

Markov Chain

Seyed Rumaiz

2026-04-19

Markov Chain Model

A discrete-time Markov chain is a stochastic process $\{X_n\}_{n \geq 0}$ that satisfies the Markov property:

$$P(X_{n+1} = j \mid X_n = i, X_{n-1}, \dots, X_0) = P(X_{n+1} = j \mid X_n = i)$$

This means that the future state depends only on the current state.

Transition Matrix

The system is described by a transition probability matrix P :

$$P = \begin{bmatrix} p_{11} & p_{12} & \cdots & p_{1k} \\ p_{21} & p_{22} & \cdots & p_{2k} \\ \vdots & \vdots & \ddots & \vdots \\ p_{k1} & p_{k2} & \cdots & p_{kk} \end{bmatrix}$$

where

$$p_{ij} = P(X_{n+1} = j \mid X_n = i), \quad \sum_{j=1}^k p_{ij} = 1$$

State Distribution

Let the initial distribution be:

$$\pi^{(0)} = (\pi_1, \pi_2, \dots, \pi_k)$$

Then the distribution after one step is:

$$\pi^{(1)} = \pi^{(0)} P$$

After n steps:

$$\pi^{(n)} = \pi^{(0)} P^n$$

Steady-State Distribution

The steady-state (stationary) distribution π satisfies:

$$\pi P = \pi$$

with the constraint:

$$\sum_{i=1}^k \pi_i = 1$$

This is the long-run behaviour of the Markov chain.

Implementation

Lets implement the Markov Chain with some examples! We will use the `markovchain` package, along with the `expm` package for matrix power.

```
# Install packages
options(repos = c(CRAN = "https://cloud.r-project.org"))
install.packages("markovchain")

##
## The downloaded binary packages are in
## /var/folders/_n/vhgg1kr10xj5cl8154sj_qgc0000gn/T//RtmpNxYgVx/downloaded_packages
install.packages("expm")

##
## The downloaded binary packages are in
## /var/folders/_n/vhgg1kr10xj5cl8154sj_qgc0000gn/T//RtmpNxYgVx/downloaded_packages
# Load libraries
library(markovchain)

## Loading required package: Matrix
## Package: markovchain
## Version: 0.10.3
## Date: 2026-02-02 06:30:37 UTC
## BugReport: https://github.com/spedygiorgio/markovchain/issues
library(expm)

##
## Attaching package: 'expm'
## The following object is masked from 'package:Matrix':
##
## expm
# Define transition matrix
states <- c("Sunny", "Cloudy", "Rainy")

transitionMatrix <- matrix(c(
  0.7, 0.2, 0.1,
  0.3, 0.4, 0.3,
  0.2, 0.3, 0.5),
```

```

nrow=3, byrow=TRUE)

transitionMatrix

##      [,1] [,2] [,3]
## [1,]  0.7  0.2  0.1
## [2,]  0.3  0.4  0.3
## [3,]  0.2  0.3  0.5

# Create a Markov Chain object
weatherMC <- new("markovchain",
                states = states,
                transitionMatrix = transitionMatrix,
                name = "Weather")

weatherMC

## Weather
## A 3 - dimensional discrete Markov Chain defined by the following states:
## Sunny, Cloudy, Rainy
## The transition matrix (by rows) is defined as follows:
##      Sunny Cloudy Rainy
## Sunny    0.7    0.2    0.1
## Cloudy    0.3    0.4    0.3
## Rainy     0.2    0.3    0.5

# Predict next state probabilities
initialState <- c(1,0,0) # Sunny

nextDay <- initialState %*% transitionMatrix
nextDay

##      [,1] [,2] [,3]
## [1,]  0.7  0.2  0.1

# Weather after 3 days
transitionMatrix3 <- transitionMatrix %^% 3
transitionMatrix3

##      [,1] [,2] [,3]
## [1,] 0.510 0.268 0.222
## [2,] 0.426 0.292 0.282
## [3,] 0.396 0.298 0.306

# Long run probabilities
steadyStates(weatherMC)

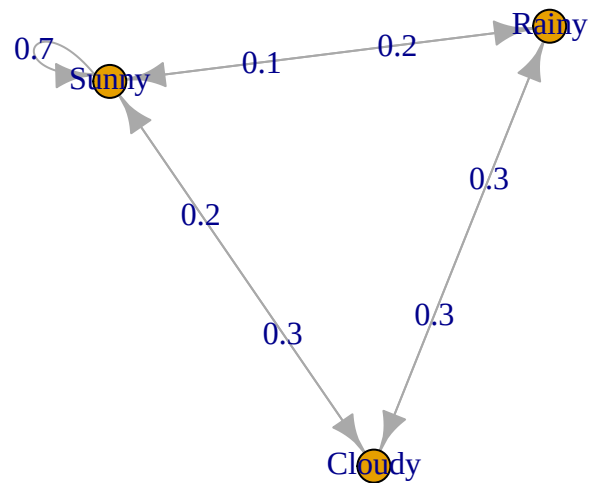
##      Sunny    Cloudy    Rainy
## [1,] 0.4565217 0.2826087 0.2608696

# Simulate a Markov Chain sequence (generate weather for 10 days)
set.seed(123)
weatherSimulation <- rmarkovchain(n=10, object=weatherMC, t0="Sunny")
weatherSimulation

## [1] "Sunny" "Cloudy" "Sunny" "Cloudy" "Rainy" "Rainy" "Cloudy" "Rainy"
## [9] "Cloudy" "Sunny"

```

```
# Plot the Markov Chain
plot(weatherMC)
```



```
# Find the probability after 2 steps starting from Rainy
initial <- c(0,0,1) # Rainy

P2 <- transitionMatrix %^% 2

result <- initial %*% P2
result

##      [,1] [,2] [,3]
## [1,] 0.33 0.31 0.36
```